map reduce, but in this case, we will use a customized version.

#3 Initialize `numShards` mappers, each will run the `canopyClustering` method and will hold a copy of one of the dataset's shards. The mappers will be alive (and hold the same copy of the input) for the whole lifetime of this method.

#4 Initialize a single reducer, that will run `canopyClustering` on the set of all centroids returned by all mappers. This node also will be alive for the whole method's lifetime, but it won't hold any data.

#5 Run MapReduce, using the mappers and reducers already created. Mappers will receive the current values for the thresholds (in addition, each mapper already hold one shard of the dataset points). The single reducer will read their input (the canopy centroids selected for each shard) from the mappers, and returned a refined list of centroids.

#6 Update the method run in the mapper nodes. Ideally, the same machines can be reused, so we don't need to shard the dataset again nor to transfer the shards to new machines (this, however, might not be possible in all MapReduce implementations). The new method to run is a simple classification step going through all points, and for each point check which centroids are within distance `T1+T2`. Each mapper will take the list of canopy centroids as input (plus its shard of initial dataset).

#7 Initialize a set of reducers, one for each canopy created from the first MapReduce job. Each reducer will get the points assigned to a single specific canopy (centroid), and will return the list of all points assigned to that canopy.

#8 Run the new MapReduce job, and get the final list of canopies (following the same order as `centroids`).